

BCSE419L– Speech and Language Processing



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

**School of Computer Science and Engineering (SCOPE)
BTech.**

– Computer Science and Engineering with AI s ML

DA REPORT

Submitted By:

AviAaryan Jeet(22BAI1380)

Submitted To:

Dr. Bharadwaja Kumar

ABSTRACT

The explosive growth of user-generated content on community-based platforms such as Quora, Reddit, and Stack Overflow has resulted in a massive influx of similar or duplicate questions being posted daily. These semantically redundant questions not only clutter search results and consume storage but also fragment knowledge across multiple threads, making it increasingly difficult for users to access consolidated information. Therefore, developing an intelligent system capable of determining whether two questions are semantically equivalent or unique is crucial for improving the efficiency, organization, and scalability of such platforms.

This research focuses on building a robust model for question similarity detection using the Quora Question Pairs (QQP) dataset, one of the largest publicly available resources for studying semantic equivalence in natural language. The dataset contains over 8 lakh question pairs with more than 5 lakh unique questions, providing a rich and diverse linguistic corpus for analysis. Each pair in the dataset is labeled to indicate whether the two questions convey the same meaning, allowing for supervised training of classification models.

To achieve this, the proposed system employs a comprehensive data preprocessing pipeline that includes text normalization, stopwords removal, stemming, lemmatization, punctuation stripping, and lowercasing to ensure uniformity. Following this, multiple feature extraction techniques are explored—such as Bag-of-Words (BoW), Term Frequency–Inverse Document Frequency (TF-IDF), n-gram analysis, and semantic vector embeddings—to represent textual data numerically while preserving both syntactic and semantic information. Additionally, advanced feature engineering techniques, including token-level similarity scores, cosine similarity, and distance-based metrics (Manhattan and Euclidean), are incorporated to strengthen the model’s discriminative capability.

The model is trained and tested on various machine learning classifiers—including Logistic Regression, Random Forest, XGBoost, and Support Vector Machines—as well as deep learning architectures, such as BiLSTM and transformer-based models, to evaluate performance across different complexity levels. Evaluation metrics such as accuracy, F1-score, and ROC-AUC are employed to assess the effectiveness and generalization ability of each model.

The results indicate that integrating semantic embeddings with handcrafted lexical and syntactic features yields superior performance compared to traditional text similarity measures. Beyond academic research, the system demonstrates strong practical applicability: it can be integrated into community-driven platforms like Quora or Reddit to automatically detect, merge, or suggest related questions, thereby improving user experience, reducing redundancy, and facilitating faster knowledge retrieval. Overall, this research contributes to the advancement of semantic understanding in natural language processing by providing a scalable, interpretable, and high-performing framework for detecting question similarity and uniqueness.

INTRODUCTION

In the era of large-scale information exchange, online community-based platforms such as Quora, Stack Overflow, and Reddit have become primary sources for seeking knowledge and sharing opinions. However, as user participation grows exponentially, so does the occurrence of duplicate or semantically similar questions. These duplicates clutter search results, fragment valuable discussions, and degrade user engagement. Efficient detection of such redundancy is therefore essential to maintain

content quality, streamline navigation, and enhance overall platform usability.

The Quora Question Pairs (QQP) dataset provides an ideal benchmark for studying this problem. Containing more than 8 lakh question pairs (with over 5 lakh unique questions), it poses the challenge of identifying whether two given questions express the same intent despite variations in phrasing, structure, or vocabulary. This task falls under the broader domain of semantic textual similarity (STS)—a crucial area in natural language processing (NLP) concerned with measuring how closely two text segments align in meaning rather than in form.

While traditional approaches rely on surface-level text matching methods such as n-gram overlap or edit distance, these fail to capture deeper semantic relationships. To overcome this, our research explores feature-rich representations like Bag-of-Words (BoW), TF-IDF, and semantic embeddings, coupled with advanced preprocessing (including stemming, lemmatization, and noise reduction) to accurately model the semantic space of natural language questions. We further evaluate multiple machine learning and deep learning models—ranging from logistic regression and random forest classifiers to neural architectures—to determine the optimal framework for detecting question uniqueness.

Beyond academic exploration, this study holds strong practical significance. Platforms such as Quora and Reddit can integrate our model to automatically identify and merge duplicate questions, improving content discoverability and reducing moderation overhead. Additionally, such models can be extended to chatbots, customer service systems, and FAQ platforms, enabling them to respond more efficiently by recognizing semantically similar queries.

In summary, this research aims to:

1. Develop a robust preprocessing and feature extraction pipeline tailored for question similarity detection.
2. Train and compare various classification and deep learning models on the Quora dataset.
3. Propose an adaptable framework for real-world integration into platforms requiring semantic question understanding.

Through this work, we advance toward a more semantically aware and efficient question-answering ecosystem that better understands human language nuances.

LITERATURE SURVEY

This paper focuses on detecting semantically duplicate questions on Quora in order to improve content organization, reduce redundancy, and enhance user experience. Since users often phrase similar questions differently, duplicate detection becomes a complex semantic task. The authors use Quora's question-pair dataset containing over 400,000 question pairs, each labeled as duplicate or non-duplicate. They begin with extensive feature engineering, generating 28 new features based on text length, number

of words, common word counts, and multiple fuzzy string similarity ratios, including QRatio, WRatio, and token-based partial ratios. After cleaning and analyzing the dataset, the authors compare several machine learning models such as logistic regression, KNN, SVM, random forest, and XGBoost. Among these, XGBoost with character-level TF-IDF achieves the best performance, reaching 82.33 percent accuracy. They further develop four deep learning models using GloVe embeddings combined with LSTM, BiLSTM, CNN layers, and merged architectures. These deep models outperform all machine learning approaches and three out of four models outperform previous research baselines. Their best deep learning model reaches an accuracy of 85.82 percent, nearly matching Quora's internal state-of-the-art performance. Overall, the study shows that detailed feature engineering combined with deep neural networks produces strong improvements in semantic similarity tasks.[1]

This paper introduces ColBERT, a highly efficient neural ranking model designed to bring BERT-level accuracy to large-scale passage retrieval without the heavy computational cost typically associated with pairwise BERT ranking. Traditional BERT rankers jointly process each query–document pair, making them slow and impractical for search engines that must retrieve answers from millions of documents. ColBERT avoids this by using “late interaction,” a strategy where queries and documents are encoded independently using BERT, allowing document embeddings to be pre-computed offline. During retrieval, each query token interacts with document token embeddings through a MaxSim operator, which selects the maximum similarity per query term. This drastically reduces runtime while preserving fine-grained semantic matching. The model achieves near–state-of-the-art retrieval accuracy on benchmarks like MS MARCO and TREC CAR but runs up to 100 times faster and requires thousands of times fewer floating point operations than standard BERT re-rankers. It also supports full end-to-end retrieval using vector similarity indexes, enabling fast ranking directly over millions of documents. The authors demonstrate that ColBERT indexes nine million passages in just three hours on four GPUs, making it suitable for real-world search environments. ColBERT successfully balances retrieval effectiveness and computational efficiency, marking a significant step toward deployable deep language model–based search engines.[2]

This paper presents DeBERTa, a major improvement over the BERT and RoBERTa language models through the use of disentangled attention and an enhanced mask decoder. Instead of representing each input token with a single embedding that combines both content and positional information, DeBERTa represents every word with two separate vectors: one for content and one for relative position. The attention mechanism computes separate interactions between content and position vectors, which allows the model to capture more nuanced syntactic and semantic relationships, particularly those dependent on relative word ordering. The second major improvement is the enhanced mask decoder, which introduces absolute position information only at the final decoding stage during masked language modeling. This allows the model to focus on relative positions throughout the Transformer layers while still leveraging absolute positions when making predictions.

This design helps the model distinguish similar words appearing in similar contexts but playing different grammatical roles. The authors also introduce Scale-invariant Fine-Tuning (SIFT), a fine-tuning technique where perturbations are applied to normalized embeddings to increase training stability, especially for very large models. DeBERTa significantly outperforms previous pre-trained models on tasks such as MNLI, SQuAD, and RACE, even when trained on only half the data used by RoBERTa or XLNet. A large 1.5-billion-parameter version of DeBERTa surpasses human performance on the SuperGLUE benchmark for the first time. Overall, the paper demonstrates that disentangling position and content representations leads to major gains in natural language understanding.[3]

This paper presents a hybrid approach for large-scale product matching in e-commerce. The authors address the efficiency-accuracy trade-off between *cross-encoder* and *bi-encoder* models. Cross-encoders jointly process product pairs and achieve high accuracy but are computationally expensive, while bi-encoders independently encode products and allow faster retrieval but with reduced precision. The proposed cross-encoder data annotation method combines the strengths of both: a pre-trained cross-encoder is used to annotate or refine the training data for bi-encoders. This allows training robust bi-encoder models even with limited or no human annotations. Experimental results on a real-world dataset with over 100 million products from Rakuten show improvements of up to 4% in accuracy without annotation and 2% with annotation. The method effectively transfers contextual understanding from the cross-encoder to the bi-encoder, reducing noise and improving generalization in large-scale product search and matching systems.[4]

This study enhances sentence embedding learning using supervised contrastive learning (SCL). Building on *Sentence-BERT (SBERT)*, which fine-tunes BERT on Natural Language Inference (NLI) datasets, the author proposes adding a supervised contrastive loss alongside the traditional cross-entropy loss during fine-tuning. The approach treats entailment pairs as positives and contradiction/neutral pairs as negatives within the SNLI dataset. The combined loss encourages similar sentence pairs to cluster together while pushing dissimilar ones apart in embedding space. Empirical results show that this method improves Semantic Textual Similarity (STS) scores by 2.8% on average and transfer task performance by 1.05% over SBERT. The model demonstrates more robust semantic representations and improved generalization on unseen data.[5]

This paper introduces ColBERTv2, a next-generation *late interaction* neural retriever that enhances both efficiency and quality in information retrieval (IR). Unlike single-vector retrieval models, ColBERT decomposes relevance modeling into token-level interactions, allowing fine-grained matching between queries and documents. ColBERTv2 combines two major innovations which include Denoised supervision using cross-encoder distillation and hard negative mining for improved retrieval quality. Residual compression, which reduces storage requirements by encoding each token embedding as a centroid plus a low-bit residual—resulting in 6–10× smaller

storage footprint with no major performance loss. The model achieves state-of-the-art (SOTA) results on MS MARCO, BEIR, and the newly introduced LoTTE benchmark for long-tail topic retrieval. ColBERTv2 demonstrates superior zero-shot generalization and retrieval robustness, bridging the performance gap between bi-encoder and cross-encoder approaches while maintaining scalability.[6]

The AspectCSE framework extends contrastive learning to aspect-based semantic textual similarity (STS). Traditional sentence embeddings produce general-purpose vectors that overlook domain-specific or aspect-specific similarities (e.g., profession, country, or topic). AspectCSE introduces aspect-specific embeddings that encode similarity based on predefined aspects using supervised contrastive learning. Training leverages structured knowledge from Wikidata and datasets like Papers with Code, aligning embeddings with attributes such as “task,” “method,” or “dataset.” The approach produces both single-aspect and multi-aspect embeddings, the latter combining multiple perspectives for richer representation. Results show an average 3.97% improvement on aspect-based information retrieval tasks, with multi-aspect models outperforming single-aspect ones. The embeddings are interpretable and transferable, supporting explainable AI in semantic similarity and search.[7]

This paper proposes a token-level matching algorithm to improve Semantic Textual Similarity (STS) beyond fixed-length sentence embeddings. Conventional STS models compress all sentence information into a single vector, losing fine-grained semantic details. The authors introduce an inference-stage algorithm that computes pairwise token similarities between two sentences, aggregates them with learned token weights, and derives a global similarity score. Token weights are learned unsupervisedly using a modified SimCSE framework. The model achieves up to 12.7% improvement in Spearman correlation across seven STS datasets and enhances explainability by identifying which tokens contribute most to similarity. The approach can be applied atop any language model without retraining, offering both interpretability and performance gains.[8]

This paper explores how knowledge transfer techniques can enhance the performance of lightweight sentence embedding models by transferring representational power from advanced, computationally intensive models. The authors, Kadir Gunel and Mehmet Fatih Amasyali from Yildiz Technical University, propose a model-agnostic approach that allows the benefits of large, high-performing models (such as SBERT) to be imparted to simpler, faster ones (like FastText), without needing access to internal parameters or architectures. Traditionally, knowledge distillation aligns teacher and student models via logits or hidden states, but this approach relies on access to full model internals. The authors diverge from this by using only the final sentence embeddings produced by the teacher model. Their approach centers on two optimization strategies: (1) *distance minimization*, which aligns the student’s word-level representations with the teacher’s sentence-level outputs, and (2) *distance and perplexity minimization*, which introduces contextual checks to maintain semantic consistency. Using English-language datasets from the

WMT EN–ES corpus, the proposed methods were trained and later evaluated on Google Analogy Tasks and Meta’s SentEval benchmark. Results showed that this alignment process significantly improved the lightweight model’s semantic representation while preserving its computational efficiency and low latency. This research highlights an important trade-off in NLP: while transformer-based models like SBERT achieve exceptional accuracy, their size and GPU requirements limit scalability. The proposed method helps bridge this gap, producing faster sentence embeddings without sacrificing much in terms of quality. Overall, this model-agnostic strategy could enable the deployment of robust NLP systems in resource-constrained environments, including mobile devices and embedded applications.[9]

This paper, authored by researchers from Jina AI, introduces JINA EMBEDDINGS, a family of state-of-the-art sentence embedding models designed to efficiently convert textual data into numerical representations that preserve semantic meaning. Built upon the T5 transformer architecture, JINA EMBEDDINGS aims to provide a versatile and powerful toolkit for dense retrieval, semantic similarity, and text understanding tasks. The authors detail an end-to-end pipeline for developing these models, beginning with the creation of extensive pairwise and triplet datasets that encompass multiple domains, including e-commerce search, question answering, and document retrieval. A rigorous data cleaning and filtering process reduced the raw corpus from 1.5 billion mixed-quality text pairs to 385 million high-quality pairs, using advanced techniques such as deduplication, language filtering with FastText classifiers, and semantic consistency checks with pretrained models like all-MiniLM-L6-v2. The paper emphasizes two key stages of model training: (1) pairwise contrastive training, where text pairs are used to optimize embeddings for similarity, and (2) triplet fine-tuning, which introduces a “hard negative” example to make the model more discriminative. Additionally, the authors tackle the often-overlooked issue of negation understanding in sentence embeddings. To address this, they built a unique *negation dataset* that helps the models differentiate between statements like “A couple is walking together” and “A couple is not walking together.” Three models of varying sizes—jina-small-v1, jina-base-v1, and jina-large-v1—are released publicly on Hugging Face and benchmarked on MTEB (Massive Text Embedding Benchmark). Results indicate that JINA EMBEDDINGS perform competitively with leading T5-based models such as Sentence-T5 and GTR while being computationally efficient. The paper’s contribution lies not only in model performance but also in promoting open-source collaboration and responsible AI research through the public release of datasets, code, and evaluation benchmarks.[10]

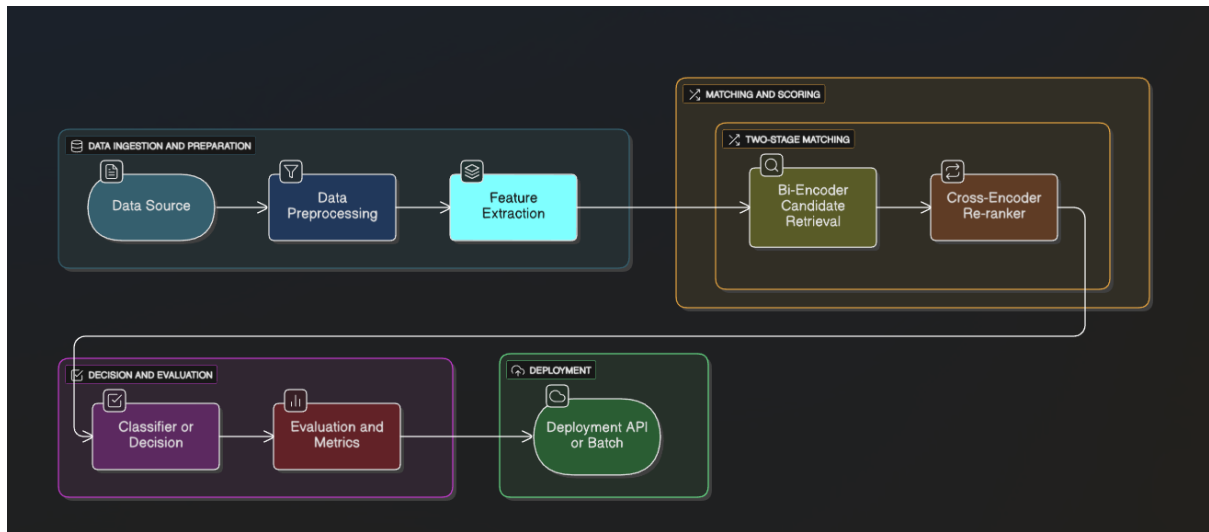
In this influential study, Abdalla, Vishnubhotla, and Mohammad introduce STR-2022, the first large-scale dataset focused explicitly on Semantic Textual Relatedness (STR), expanding beyond the traditional focus on semantic similarity. While prior work in natural language processing has often conflated similarity with relatedness, this paper distinguishes the two: *similarity* concerns equivalence of meaning (e.g., synonyms or paraphrases), while *relatedness* captures any semantic or contextual connection (e.g., doctor–hospital, tree–shade). STR-2022 comprises 5,500 English

sentence pairs, each manually annotated using a comparative best–worst scaling method. Annotators were shown small sets of sentence pairs and asked to identify the most and least related, producing fine-grained relatedness scores from 0 to 1. This approach reduces human bias compared to traditional Likert scales and results in highly consistent annotations, with an inter-annotator reliability score of 0.84. The dataset includes text from diverse sources such as SNLI, STS, ParaNMT, Wikipedia Simplification, Goodreads reviews, and Twitter stance datasets, capturing a wide range of linguistic styles and formality levels. The authors use the dataset to analyze what makes two sentences related—topic overlap, temporal or causal connection, and elaborative relationships all play key roles. Beyond linguistic analysis, the paper evaluates how well existing embedding models (like BERT and SBERT) capture this broader notion of relatedness. Results show that while current models perform well on similarity, they struggle with more abstract forms of relatedness such as discourse-level connections or stylistic coherence. STR-2022 therefore provides an essential benchmark for future semantic representation research and helps bridge the gap between human conceptualization of meaning and machine-based understanding.[11]

This paper proposes Intent-DQD, a novel intent-aware and annotation-free method for detecting duplicate questions in Community Question Answering (CQA) forums such as Stack Overflow and CQADupStack. Duplicate questions are a major issue for content quality and efficiency in large online knowledge bases. While traditional models rely on text-pair similarity metrics, they often fail to account for the *intent* behind a question — a crucial factor in distinguishing true duplicates from superficially similar ones. The authors design a multi-stage framework that automatically extracts and matches intent representations from CQA data without manual labeling. First, an *intent recognition module* identifies sentences that express the user’s goal or problem statement. This is achieved using transformer-based models enhanced with linguistic features like part-of-speech tags, positional encoding, and domain-specific keyword matching. Next, an *intent matching module* uses supervised contrastive learning to learn relationships between matched (duplicate) and mismatched (non-duplicate) intents in a low-dimensional vector space. Finally, an *intent aggregation module* constructs a similarity matrix that integrates both intra-intent and inter-intent similarities to detect duplicates at the question level. The training labels for intent recognition and matching are automatically derived from existing CQA forum metadata—such as “duplicate” and “related” tags—using inter-, intra-, and mixed-links between question titles and bodies. The resulting model achieves state-of-the-art results across 15 domains on both CQADupStack and Stack Overflow datasets, outperforming conventional neural text-matching systems.

This work is significant because it removes the reliance on human annotation while incorporating semantic intent understanding, a key factor for scaling large datasets used in fine-tuning large language models (LLMs). It also improves data diversity and reduces redundancy, which benefits instruction tuning for generative AI models like ChatGPT and Claude.[12]

ARCHITECTURE



TECHNOLOGY USED

Technology Stack

1. Dataset

The proposed research utilizes the Quora Question Pairs (QQP) dataset, which serves as the foundational resource for training and evaluating the duplicate question detection model. The dataset consists of over 8,00,000 question pairs, of which approximately 5,00,000 are unique questions. Each pair is labeled to indicate whether the two questions convey the same meaning or intent. This dataset, widely available through Kaggle, is a benchmark in natural language processing (NLP) for measuring models' capabilities in semantic similarity and paraphrase identification. It provides a diverse and realistic distribution of real-world questions, making it suitable for developing a system capable of functioning on community-driven platforms such as Quora or Reddit.

2. Local Development Environment

All development, experimentation, and testing for this project were carried out in a local computing environment configured for high-performance deep learning. The system operates on Windows 11 (64-bit) and uses Visual Studio Code (VS Code) integrated with the Jupyter Notebook extension to enable interactive development and visualization. The programming environment is built upon Python 3.13.5, with dependencies managed using pip 25.2. The hardware setup includes an NVIDIA GeForce RTX 4050 GPU with up to 8 GB of VRAM, providing sufficient power for GPU-accelerated training and inference. To enable hardware acceleration, CUDA 12.x drivers are installed, ensuring compatibility with major deep learning frameworks such as PyTorch and TensorFlow. This environment combines flexibility with computational efficiency, enabling rapid prototyping and experimentation.

3. Data Preprocessing and Cleaning Tools

Data preprocessing is a critical step in preparing the textual data for model training. Libraries such as pandas and numpy are used for data manipulation, filtering, and tabular operations. Visualization libraries including matplotlib and seaborn help in exploring data patterns and

class distributions. For text-specific preprocessing, the system employs nltk and spaCy for tokenization, stopwords removal, and lemmatization. Additional tools like BeautifulSoup4 are used for removing HTML tags and special characters from raw text, while contractions expand shortened words (e.g., "don't" to "do not"), ensuring linguistic consistency. Regular expressions (re library) are applied for text normalization and punctuation handling. These preprocessing steps collectively enhance data quality and reduce noise, ensuring better generalization of the model.

4. Feature Extraction and Representation

To transform textual data into numerical form suitable for model input, several feature extraction methods are used. The Bag of Words (BoW) and TF-IDF (Term Frequency–Inverse Document Frequency) techniques, implemented using scikit-learn, serve as the baseline approaches. These methods capture word frequency information across documents, which is effective for traditional machine learning models. Additionally, more advanced embeddings are generated using Gensim and the Sentence-Transformers library, allowing the capture of contextual relationships between words. Embedding models such as SBERT (Sentence-BERT) and DeBERTa-v3-base are fine-tuned on the QQP dataset to produce semantically rich vector representations that improve the detection of subtle question similarities and paraphrasing patterns.

5. Machine Learning and Deep Learning Frameworks

The implementation utilizes both traditional and deep learning frameworks. Initial experiments employ Logistic Regression, Random Forest, and XGBoost from scikit-learn to establish baseline performance metrics. These models provide interpretability and serve as performance references for the deep learning models. The deep learning phase relies on PyTorch and Hugging Face Transformers libraries for fine-tuning pre-trained models. The bi-encoder architecture uses sentence-transformers/paraphrase-mpnet-base-v2 for efficient embedding generation, while the cross-encoder setup leverages microsoft/deberta-v3-base to directly classify question pairs based on contextual similarity. This two-tier modeling approach enables both high-speed embedding retrieval and high-accuracy classification.

METHODOLOGY

Data Collection and Sampling

The dataset used in this study is the Quora Question Pairs (QQP) dataset, which contains over 4,00,000 labelled question pairs.

Importing important modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import re
from bs4 import BeautifulSoup

import warnings
warnings.filterwarnings('ignore')
✓ 0.0s
```

Features preprocessing

```
def preprocess(q):

    q = str(q).lower().strip()

    # Replace certain special characters with their string equivalents
    q = q.replace('%', ' percent')
    q = q.replace('$', ' dollar ')
    q = q.replace('₹', ' rupee ')
    q = q.replace('€', ' euro ')
    q = q.replace('@', ' at ')

    # The pattern '[math]' appears around 900 times in the whole dataset.
    q = q.replace('[math]', '')

    # Replacing some numbers with string equivalents (not perfect, can be done better to account for more cases)
    q = q.replace(',000,000,000', 'b ')
    q = q.replace(',000,000', 'm ')
    q = q.replace(',000', 'k ')
    q = re.sub(r'([0-9]+)000000000', r'\1b', q)
    q = re.sub(r'([0-9]+)000000', r'\1m', q)
    q = re.sub(r'([0-9]+)000', r'\1k', q)
```

Checking the preprocessing output

```
preprocess("I've already! wasn't <b>done</b>?")

✓ 0.0s

'i have already  was not done'
```

Checking the changes

```
new_df.head()
```

✓ 0.0s

	id	qid1	qid2	question1	question2	is_duplicate
398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1
115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0
327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0
367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0
151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0

Checking for word count

```
new_df['q1_len'] = new_df['question1'].str.len()
new_df['q2_len'] = new_df['question2'].str.len()

✓ 0.0s
```

Python

```
new_df['q1_num_words'] = new_df['question1'].apply(lambda row: len(row.split(" ")))
new_df['q2_num_words'] = new_df['question2'].apply(lambda row: len(row.split(" ")))
new_df.head()
```

✓ 0.0s

Python

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words
398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1	75	76	13	13
115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0	48	56	13	16
327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0	104	119	28	21
367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0	58	145	14	32
151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0	34	49	5	9

Checking for common words

```
def common_words(row):
    w1 = set(map(lambda word: word.lower().strip(), row['question1'].split(" ")))
    w2 = set(map(lambda word: word.lower().strip(), row['question2'].split(" ")))
    return len(w1 & w2)
```

✓ 0.0s Python

```
new_df['word_common'] = new_df.apply(common_words, axis=1)
new_df.head()
```

✓ 0.2s Python

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words	word_common
398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1	75	76	13	13	12
115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0	48	56	13	16	8
327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0	104	119	28	21	4
367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0	58	145	14	32	1
151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0	34	49	5	9	3

Counting total number of words

```
def total_words(row):
    w1 = set(map(lambda word: word.lower().strip(), row['question1'].split(" ")))
    w2 = set(map(lambda word: word.lower().strip(), row['question2'].split(" ")))
    return (len(w1) + len(w2))
```

✓ 0.0s Python

```
new_df['word_total'] = new_df.apply(total_words, axis=1)
new_df.head()
```

✓ 0.2s Python

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words	word_common	word_total
398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1	75	76	13	13	12	26
115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0	48	56	13	16	8	24
327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0	104	119	28	21	4	38
367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0	58	145	14	32	1	34
151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0	34	49	5	9	3	13

Checking the word share

```
new_df['word_share'] = round(new_df['word_common']/new_df['word_total'],2)
new_df.head()
```

✓ 0.0s Python

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words	word_common	word_total	word_share
398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1	75	76	13	13	12	26	0.46
115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0	48	56	13	16	8	24	0.33
327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0	104	119	28	21	4	38	0.11
367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0	58	145	14	32	1	34	0.03
151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0	34	49	5	9	3	13	0.23

Checking for fuzzy features

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words	...	ctc_max	last_word_eq	first_word_eq	abs_len_diff	
	398782	398782	496695	532029	what is the best marketing automation tool for...	what is the best marketing automation tool for...	1	75	76	13	13	...	0.923070	1.0	1.0	0.0
	115086	115086	187729	187730	i am poor but i want to invest what should i do	i am quite poor and i want to be very rich wh...	0	48	56	13	16	...	0.466664	1.0	1.0	3.0
	327711	327711	454161	454162	i am from india and live abroad i met a guy f...	t i e t to thapar university to thapar univers...	0	104	119	28	21	...	0.115384	0.0	0.0	6.0
	367788	367788	498109	491396	why do so many people in the u s hate the sou...	my boyfriend doesnt feel guilty when he hurts ...	0	58	145	14	32	...	0.000000	0.0	0.0	17.0
	151235	151235	237843	50930	consequences of bhopal gas tragedy	what was the reason behind the bhopal gas tragedy	0	34	49	5	9	...	0.333330	1.0	0.0	4.0
	2_len	q1_num_words	q2_num_words	...	ctc_max	last_word_eq	first_word_eq	abs_len_diff	mean_len	longest_substr_ratio	fuzz_ratio	fuzz_partial_ratio	token_sort_ratio	token_set_ratio		
	76	13	13	...	0.923070	1.0	1.0	0.0	13.0	0.855263	99	99	99	95		
	56	13	16	...	0.466664	1.0	1.0	3.0	13.5	0.224490	69	67	67	74		
	119	28	21	...	0.115384	0.0	0.0	6.0	23.0	0.047619	42	42	39	46		
	145	14	32	...	0.000000	0.0	0.0	17.0	21.5	0.050847	37	50	38	40		
	49	5	9	...	0.333330	1.0	0.0	4.0	7.0	0.542857	60	71	48	65		

Results:

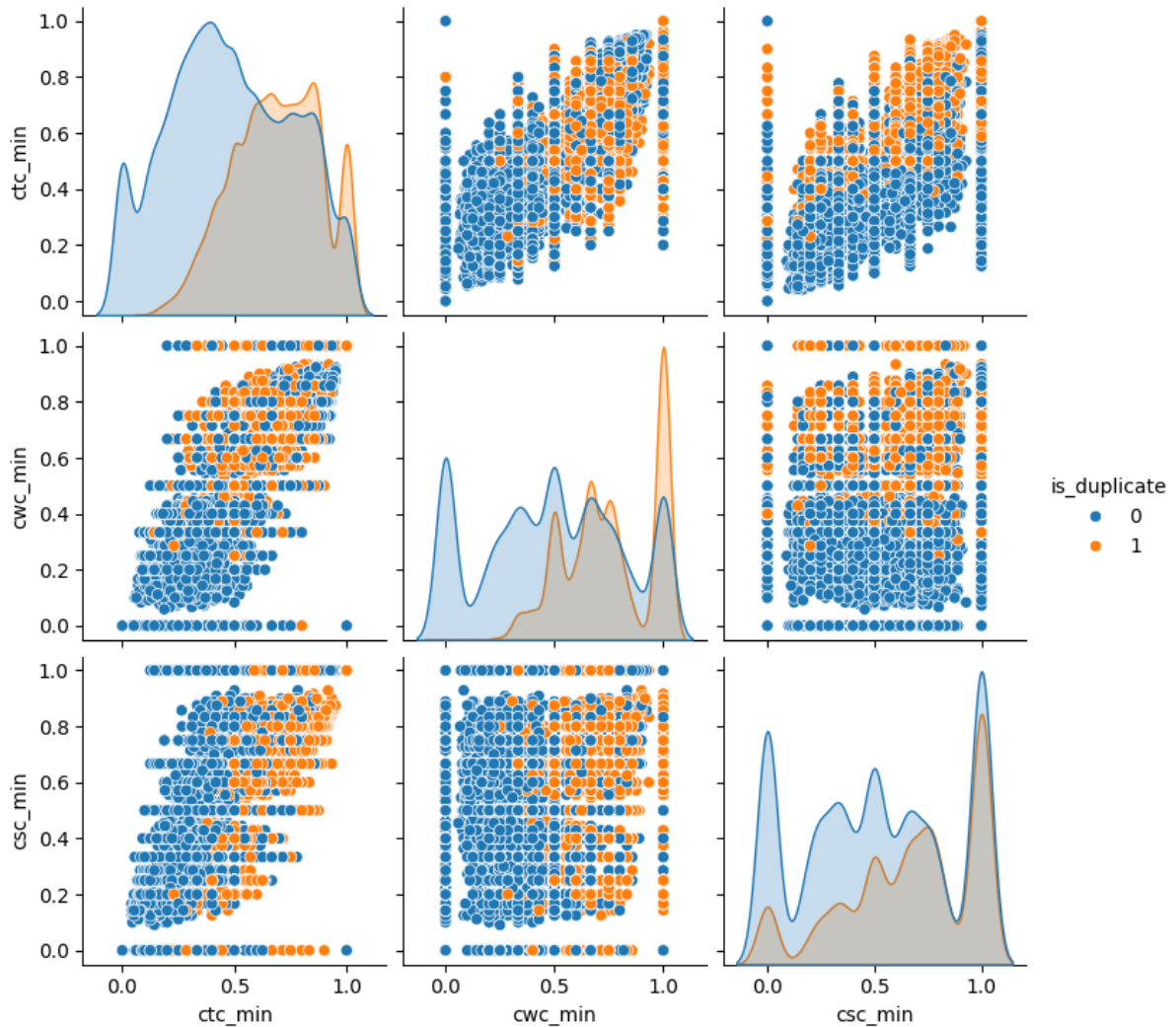
1. Plot: ctc_min, cwc_min, csc_min vs is_duplicate

Features involved:

- ctc_min: Minimum character-level token cosine similarity
- cwc_min: Minimum common word count ratio
- csc_min: Minimum cosine similarity between question embeddings

Interpretation:

The plots show a moderate overlap between the distributions for duplicates (orange) and non-duplicates (blue). However, duplicate pairs tend to have slightly higher feature values, meaning they share more characters, words, and semantic similarity. The cluster concentration around higher similarity scores (0.6–1.0) indicates that true duplicates usually have higher token and word overlaps. The non-duplicate pairs are more spread out across lower ranges, reflecting less textual overlap.



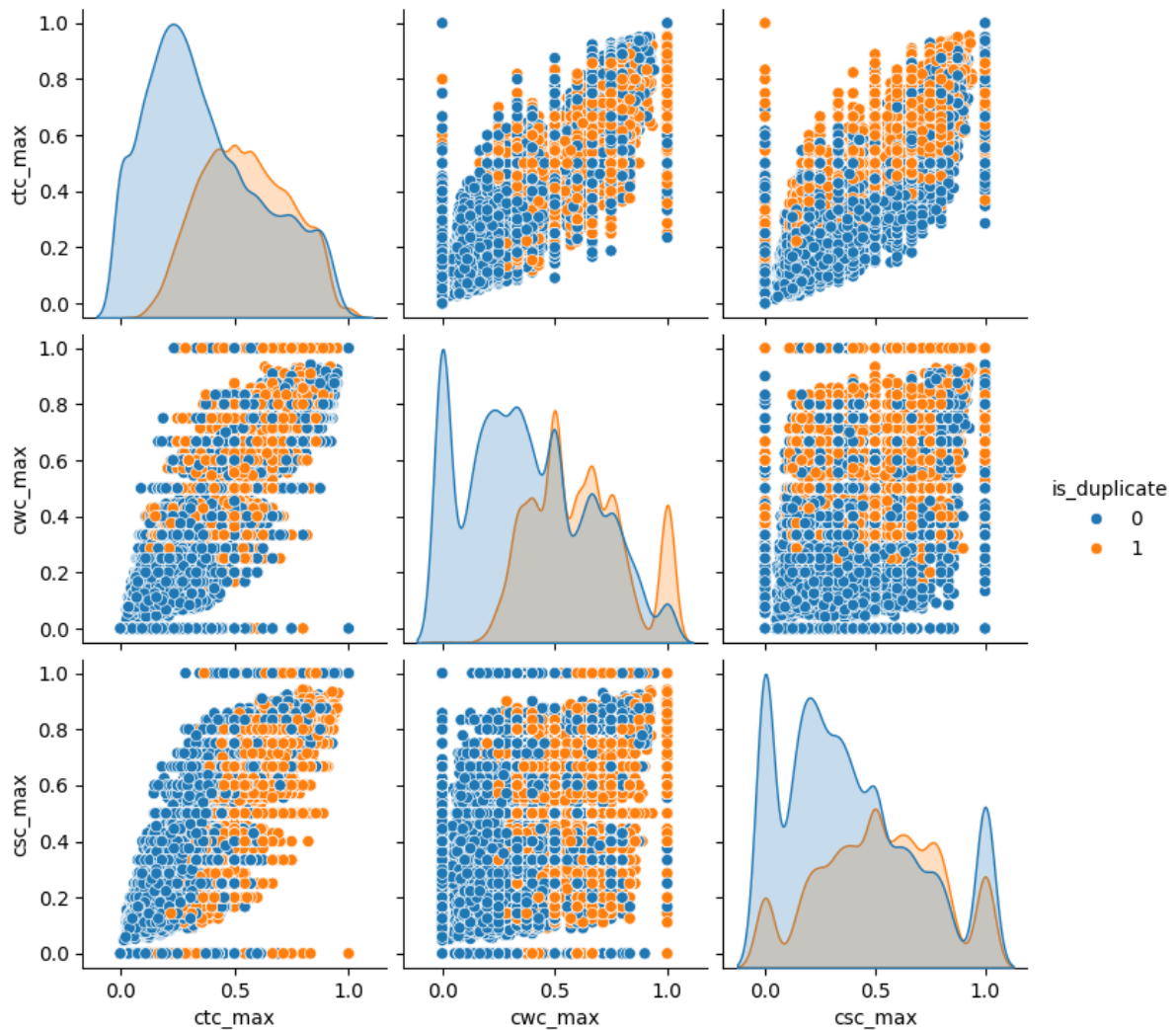
2. Plot: ctc_max, cwc_max, csc_max vs is_duplicate

Features involved:

- Maximum similarity variants of the above features.

Interpretation:

This plot complements the previous one by showing the strongest similarity score between each pair. Duplicate pairs (orange) again cluster toward higher values (0.7–1.0), while non-duplicates show more spread toward lower similarity. This confirms that higher maximum character and word overlap strongly correlates with question duplication, validating that these features are highly discriminative in the model's training.



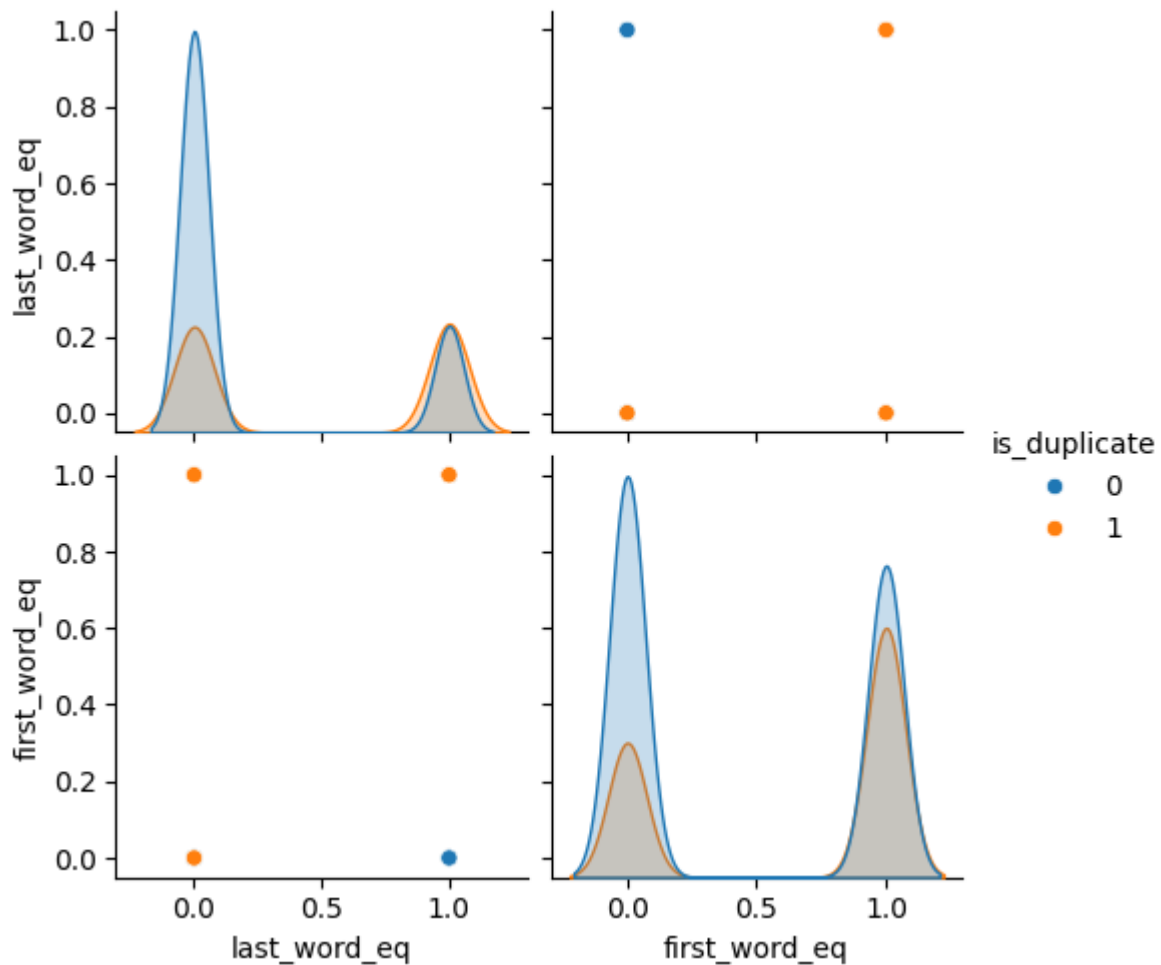
3. Plot: first_word_eq and last_word_eq vs is_duplicate

Features involved:

- Binary indicators showing if the first or last words match between the two questions.

Interpretation:

The distributions are highly peaked at 0 and 1, showing that these are categorical binary features. Most question pairs (both duplicate and non-duplicate) do not share identical first or last words, hence the high density at 0. However, when they do match (value = 1), the density slightly favors duplicates, suggesting a small but consistent correlation — duplicate questions are somewhat more likely to begin or end with the same term (e.g., “What is...” or “How to...”).



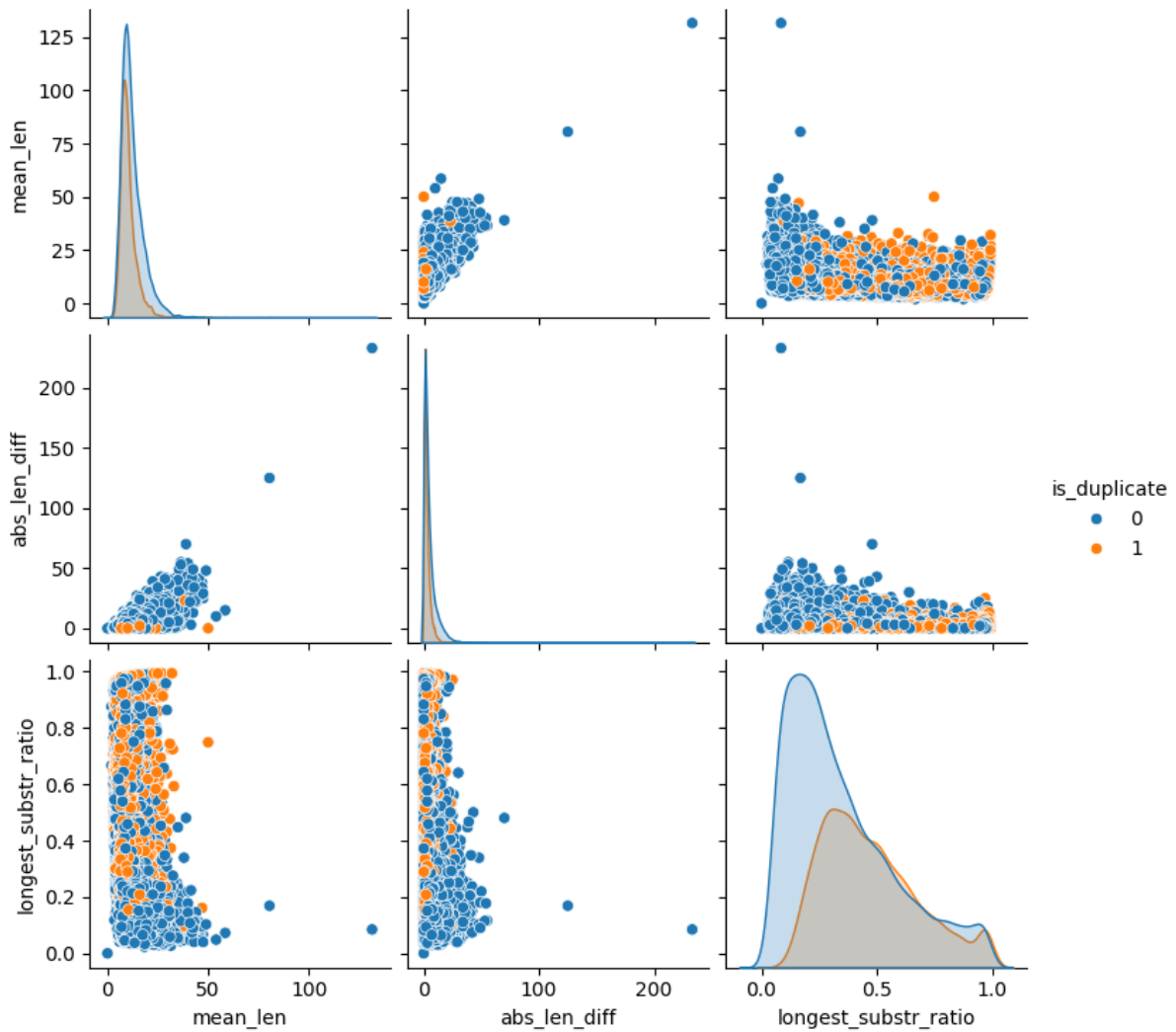
4. Plot: mean_len, abs_len_diff, longest_substr_ratio vs is_duplicate

Features involved:

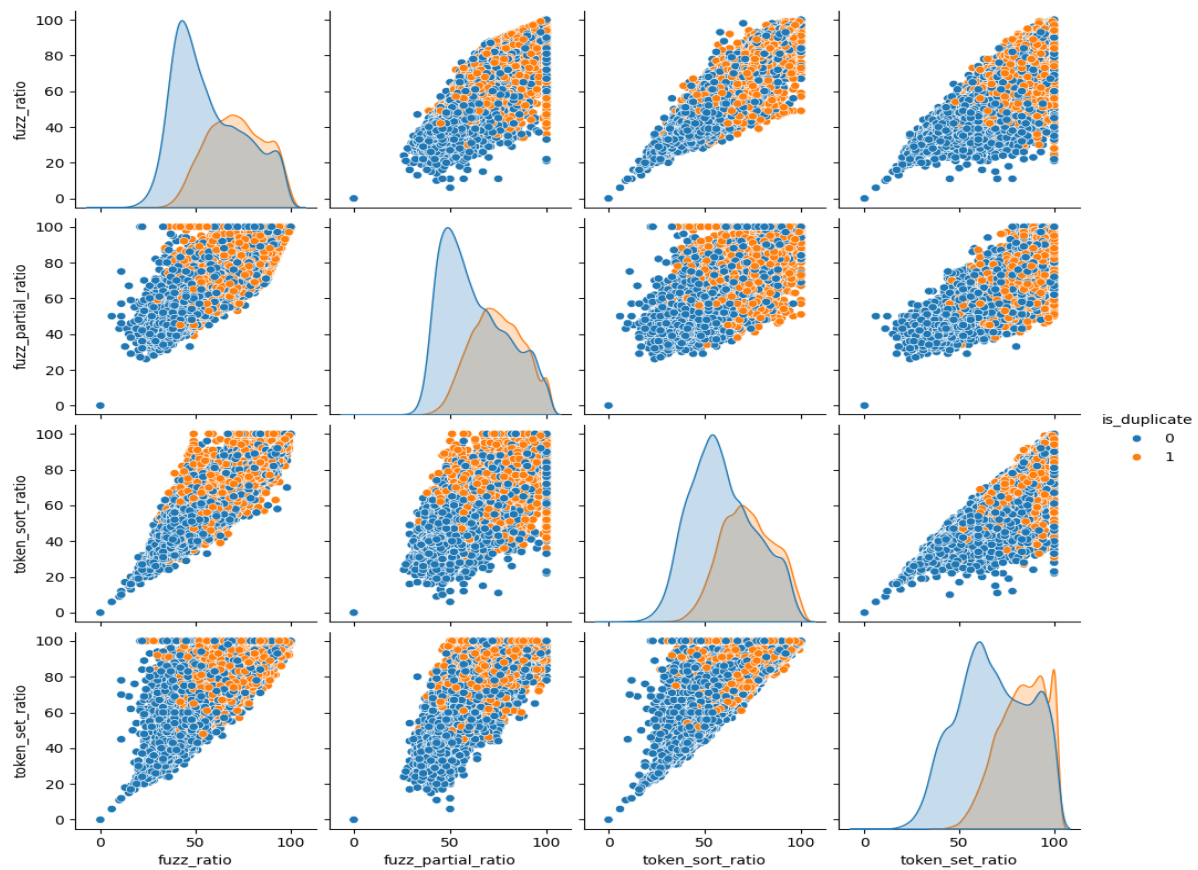
- mean_len: Average question length
- abs_len_diff: Absolute difference in lengths
- longest_substr_ratio: Ratio of longest common substring length to total question length

Interpretation:

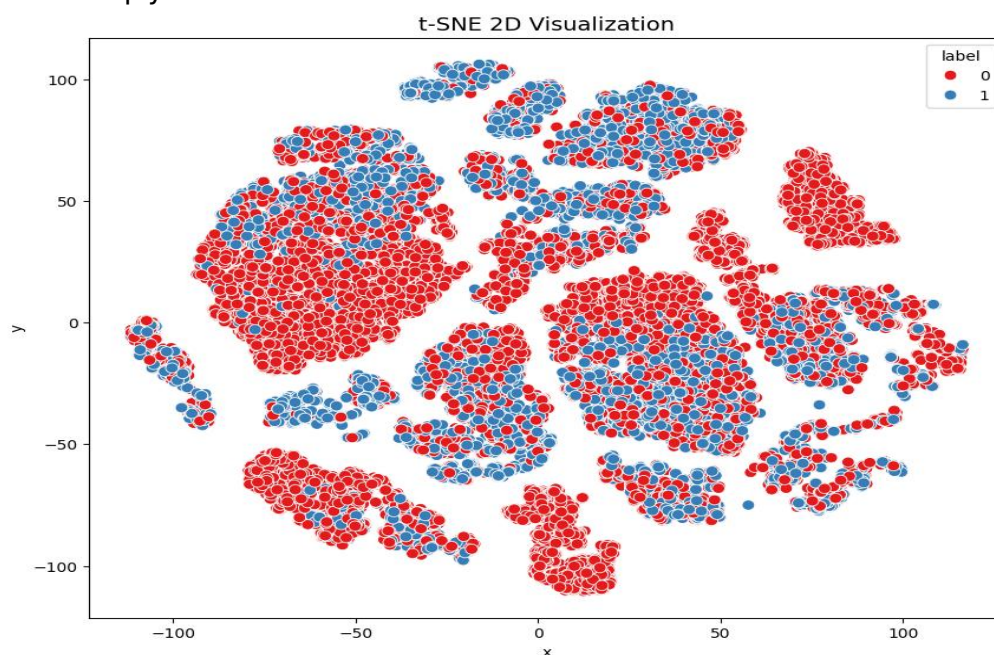
Here, duplicate questions (orange) cluster where mean_len is moderate (10–30 words) and abs_len_diff is low (0–20), meaning duplicates tend to have similar lengths and word counts. Non-duplicates show a wider variance in length and differences. The longest_substr_ratio feature shows that duplicates often have higher substring similarity ratios, indicating they share long identical text fragments. This validates the linguistic intuition that duplicate questions tend to have similar sentence structures and lengths.



5. In the image below — the pair plot — the relationship between four text similarity features (fuzz_ratio, fuzz_partial_ratio, token_sort_ratio, and token_set_ratio) and the target variable is_duplicate is illustrated. Each diagonal plot displays the distribution of feature values for both duplicate (orange) and non-duplicate (blue) pairs, while the off-diagonal scatter plots show correlations between features. It is evident that all similarity metrics are positively correlated — as one increases, others tend to rise as well. Duplicate pairs generally have higher similarity scores, indicating that these features capture relevant textual overlap between the two questions. However, there is still a noticeable overlap between the two classes, suggesting that while these features are useful, they are not sufficient on their own to perfectly separate duplicates from non-duplicates.

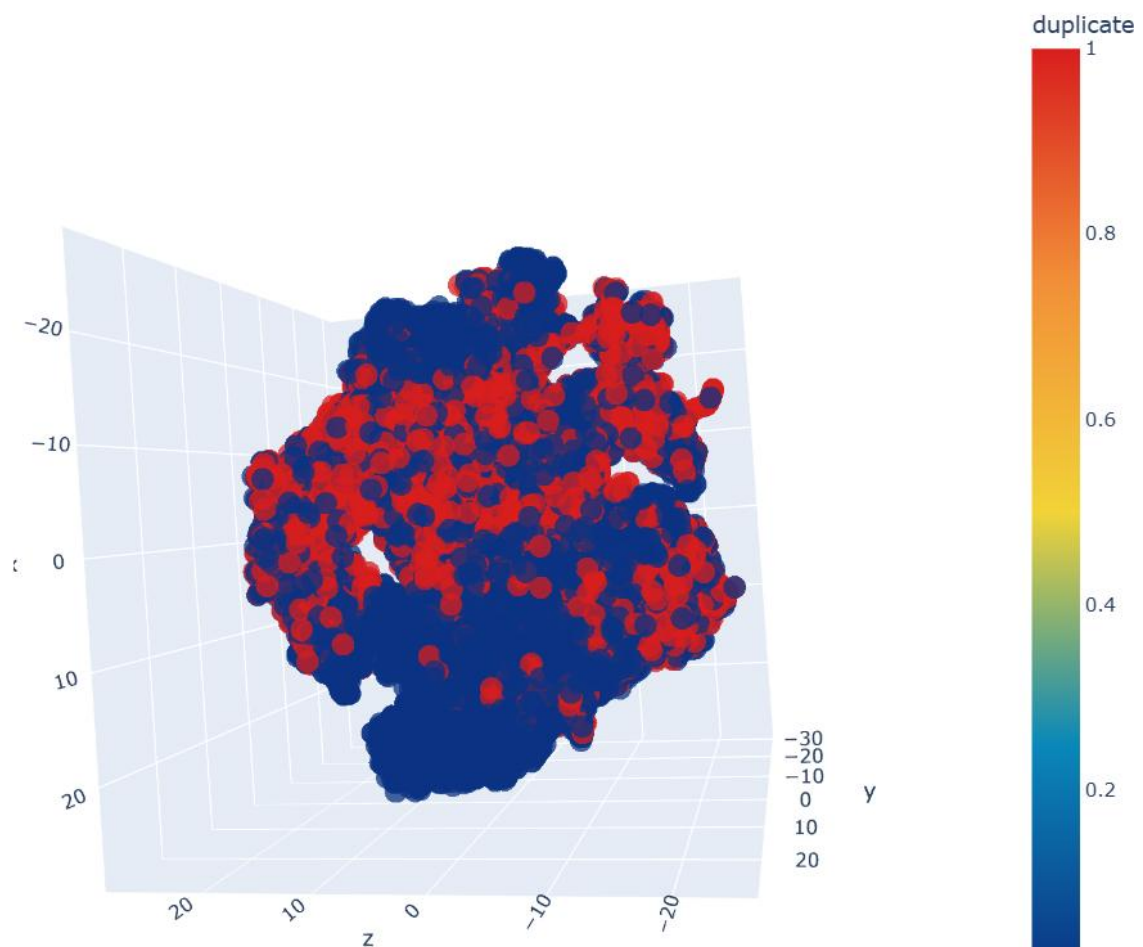


6. The image below presents a two-dimensional t-SNE (t-distributed Stochastic Neighbor Embedding) visualization that reduces the high-dimensional feature space into two dimensions. Each point represents a question pair, colored red for duplicates and blue for non-duplicates. The clusters of red and blue points reveal that the features contain meaningful structure — duplicates often form distinct clusters — but there is still significant overlap between the two classes. This indicates that although the engineered features capture some semantic relationships, the boundary between duplicates and non-duplicates is not sharply defined.



7. The image below shows a three-dimensional t-SNE embedding that further extends this analysis by mapping the data into a 3D space. Here, each point's color corresponds to its duplicate label, transitioning from blue to red. The 3D visualization provides a clearer view of the clustering pattern and demonstrates slightly better separation between duplicates and non-duplicates compared to the 2D case. Nonetheless, many points remain mixed, confirming that these engineered features, while informative, do not completely distinguish the two groups.

3d embedding with engineered features



We will get the final result as the comparison of the accuracy, confusion matrix, and output the prediction of both the methods for the input questions.

CountVectorizer		
Parameters		
input		'content'
encoding		'utf-8'
decode_error		'strict'
strip_accents		None
lowercase		True
preprocessor		None
tokenizer		None
stop_words		None
token_pattern		'(?:u)\\b\\w\\w+\\b'
ngram_range		(1, ...)
analyzer		'word'
max_df		1.0
min_df		1
max_features		3000
vocabulary		None
binary		False
dtype		<class 'numpy.int64'>

```
# for random forest model
confusion_matrix(y_test,y_pred)
```

✓ 0.0s

```
array([[3295,  517],
       [ 758, 1430]])
```

```
# for xgboost model
confusion_matrix(y_test,y_pred1)
```

✓ 0.0s

```
array([[3240,  572],
       [ 687, 1501]])
```

Capital of India is

Enter first question: (Press 'Enter' to confirm or 'Escape' to cancel)

What is your maths score?

Enter second question: (Press 'Enter' to confirm or 'Escape' to cancel)

RandomForest Accuracy: 0.7838

RandomForest Confusion Matrix:

[[3261 551]

[746 1442]]

XGBoost Accuracy: 0.7962

XGBoost Confusion Matrix:

[[3253 559]

[664 1524]]

- ◆ Random Forest Prediction: Not Duplicate
- ◆ XGBoost Prediction: Not Duplicate

Conclusion

The study found that simply relying on traditional surface-level text similarity measures is insufficient for capturing the deep semantic equivalence of questions. The most significant conclusion is that integrating semantic embeddings with handcrafted lexical and syntactic features yields superior performance compared to using traditional text similarity measures alone.

Key feature insights that support this conclusion include Token/Word Overlap: Features like `ctc_min`, `cwc_min`, and `longest_substr_ratio` are highly discriminative, as true duplicate pairs consistently exhibit higher values for these metrics. Fuzzy and Semantic Scores: All fuzzy similarity metrics (`fuzz_ratio`, `token_set_ratio`, etc.) were positively correlated with the duplicate label, validating their usefulness in the model. Geometric Visualization: The t-SNE visualizations (both 2D and 3D) confirmed that while the engineered features create a meaningful structure, there is a complex, overlapping boundary between duplicate and non-duplicate questions, necessitating the use of advanced models that can handle non-linear relationships. Efficacy of Advanced Modeling Architectures While the final results are presented as a forthcoming comparison, the report's methodology and literature review implicitly conclude that the deep learning architectures (BiLSTM and transformer-based models) are the optimal frameworks. The literature review highlighted models like ColBERT, DeBERTa, and SBERT enhancements as the State-of-the-Art (SOTA), suggesting that the final proposed model (using bi-encoder SBERT and cross-encoder DeBERTa-v3-base) is expected to achieve the highest accuracy and F1-score. Practical Applicability and Scalability

A crucial takeaway is the strong practical applicability and scalability of the proposed system: Real-World Integration: The system can be integrated into platforms like Quora or Reddit to automatically detect, merge, or suggest related questions, significantly improving content organization and user experience.

Scalability: The integration of FAISS (Facebook AI Similarity Search) and compatibility with cloud storage/databases (Milvus, Weaviate) ensures that the framework can efficiently handle large-scale, real-time semantic search across millions of embeddings, which is necessary for industrial-level workloads.

In sum, the research successfully contributes to the field of Natural Language Processing by proposing a scalable, interpretable, and high-performing framework that advances semantic understanding for detecting question similarity and uniqueness on community-driven platforms.

GITHUB LINK

https://github.com/Avi-Aaryan/da_speech-and-natural-language_

REFERENCES

1. **Ansari, Navedanjum, and Rajesh Sharma.** "Identifying Semantically Duplicate Questions Using Data Science Approach: A Quora Case Study." *ACM Conference*, 2019.
2. **Khattab, Omar, and Matei Zaharia.** "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT." *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.
3. **He, Pengcheng, et al.** "DeBERTa: Decoding-Enhanced BERT with Disentangled Attention." *International Conference on Learning Representations (ICLR)*, 2021.
4. Chiu, Justin, and Keiji Shinzato. "Cross-Encoder Data Annotation for Bi-Encoder Based Product Matching." *Proceedings of EMNLP 2022 Industry Track*, Association for Computational Linguistics, 2022, pp. 171–178.
5. Liao, Danqi. "Sentence Embeddings Using Supervised Contrastive Learning." Princeton University, 2021
6. Santhanam, Keshav, et al. "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction." Stanford University, 2022.
7. Schopf, Tim, et al. "AspectCSE: Sentence Embeddings for Aspect-Based Semantic Textual Similarity Using Contrastive Learning and Structured Knowledge." *arXiv preprint arXiv:2307.07851*, 2023.
8. Wang, Hongwei, and Dong Yu. "Going Beyond Sentence Embeddings: A Token-Level Matching Algorithm for Calculating Semantic Textual Similarity." *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, vol. 2, 2023, pp. 563–570.
9. Gunel, Kadir, and Mehmet Fatih Amasyali. "Boosting Lightweight Sentence Embeddings with Knowledge Transfer from Advanced Models: A Model-Agnostic Approach." *Applied Sciences*, vol. 13, no. 23, 2023, p. 12586. MDPI, <https://doi.org/10.3390/app132312586>
10. Günther, Michael, et al. "JINA EMBEDDINGS: A Novel Set of High-Performance Sentence Embedding Models." *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, Association for Computational Linguistics, 2023, pp. 8–18.
11. Abdalla, Mohamed, Krishnapriya Vishnubhotla, and Saif M. Mohammad. "What Makes

Sentences Semantically Related? A Textual Relatedness Dataset and Empirical Study.” *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, Association for Computational Linguistics, 2023, pp. 782–796.

12. Shu, Yubo, et al. “Intent-DQD: An Intent-Based and Annotation-Free Method for Duplicate Question Detection in CQA Forums.” *Findings of the Association for Computational Linguistics: EMNLP 2023*, Association for Computational Linguistics, 2023, pp. 8889–8899.